

Panoptic MapAnything: Unified Multi-View Consistent Panoptic Segmentation

Sai Mukkundan Ramamoorthy
Hochschule Bonn-Rhein-Sieg, Germany
University of Bonn, Germany
Email: sai.ramamoorthy@smail.inf.h-brs.de

Aaron Cuthinho
Hochschule Bonn-Rhein-Sieg, Germany
Email: aaron.cuthinho@smail.inf.h-brs.de

Abstract—Current 3D panoptic scene segmentation approaches rely on specific inputs such as point clouds, posed rgb images, etc. and demand costly test time optimization for 2D-3D alignment[18]. These models often have separate segmentation and reconstruction pipelines with additional post processing steps which results in additional overhead with efficiency losses. Our approach mitigates these deficits by effectively utilising a 3D reconstruction backbone model such as MapAnything[5] with a customised Mask2Former[3] head for multi-view panoptic segmentation thereby creating a unified metric accurate feature rich semantic 3D map. We are using the ScanNet++[17] dataset which is a large scale benchmark dataset. It provides sub-millimeter resolution laser scans with RGB-D streams for developing richer 3D panoptic segmentation and Novel View Synthesis models. We believe that our approach would show that the multi-view features from the MapAnything backbone can be effectively used by Mask2Former to provide a multi-view consistent panoptic segmentation compared to other approaches with less overhead when trained with more computing power and the whole dataset.

I. INTRODUCTION

Understanding complex 3D environments is essential for applications in robotics, augmented reality, and autonomous systems like self-driving cars, etc; where agents must accurately perceive and interact with their surroundings which requires simultaneous reconstruction of the 3D structure of environments and identifying what objects exist within them. This requires decomposing scenes into distinct object instances with semantic labels, a task known as Panoptic Segmentation[7], which unifies both instance segmentation for countable objects and semantic segmentation for amorphous regions. This dual challenge i.e. recovering geometry while recognizing semantic content, has traditionally been tackled as two separate problems, with reconstruction systems building 3D maps and segmentation systems labeling visual data independently. While significant progress has been made in 2D panoptic segmentation through transformer-based architectures; when it comes to multi-view settings, obtaining panoptic scene labels that remain consistent across viewpoints is still difficult, and existing solutions often trade off generality, compute, and integration and therefore this remains an active research challenge.

Existing approaches to multi-view 3D panoptic segmentation employ common strategies including (i) predicting labels directly on 3D inputs such as point clouds/meshes/voxels[4][9][12][13][16], and (ii) starting from images, running a 2D panoptic model per view, then lifting/fusing those predictions into 3D with NeRF[10]- or Gaussian-splatting-based[6] pipelines. While effective in some regimes, these pipelines frequently rely on accurate pose information and/or expensive test-time procedures to reconcile 2D predictions with 3D structure, and they naturally split reconstruction from segmentation into separate stages with additional overhead.

The recent emergence of powerful feed-forward reconstruction models like MapAnything marks a significant shift in 3D computer vision as it can process collections of unposed images and directly output metric-accurate 3D reconstructions by learning to share information across multiple views through transformer architectures. Critically, to achieve accurate geometric predictions, these models must develop internal representations that capture scene structure, object boundaries, and spatial relationships which is precisely the information needed for semantic understanding. This raises a fundamental question: can we leverage the rich geometric features already learned by reconstruction backbones to perform multi-view panoptic segmentation, rather than building entirely separate systems?

Our Approach, Panoptic MapAnything [1], explores that direction by integrating a metric 3D reconstruction backbone (MapAnything) with a Mask2Former-style panoptic head so that the segmentation stack can directly consume multi-view-aware features computed by the reconstruction model. In practice, we keep the MapAnything backbone frozen and introduce a trainable “panoptic DPT”[11] adapter that is initialized from MapAnything’s geometric DPT components and produces a Mask2Former-compatible multi-scale feature pyramid. These features are then processed by a Mask2Former decoder (pixel decoder + query-based transformer decoder) to predict per-view panoptic masks and classes. To encourage cross-view agreement without relying on heavy post-hoc fusion, we design a multi-view training setup that uses a reference view and

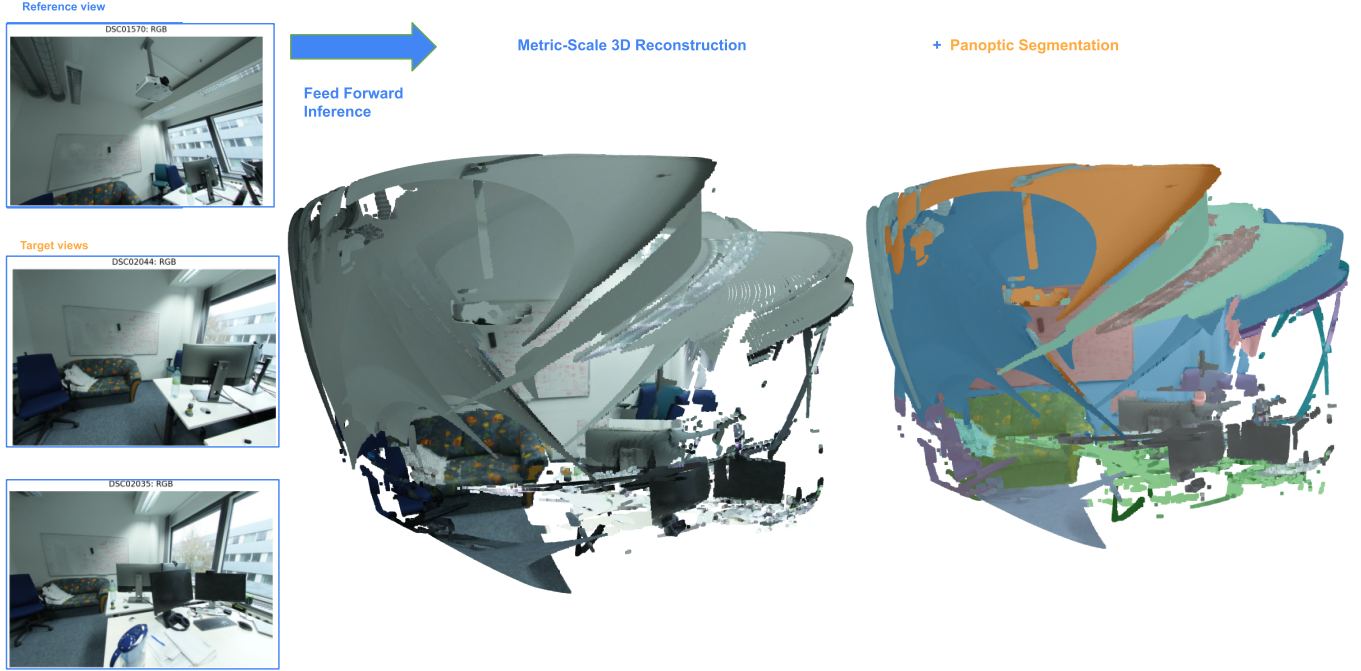


Fig. 1. **Panoptic MapAnything Overview** It is a flexible, unified feed-forward 3D reconstruction model with multi-view panoptic segmentation head, accepts multiple images to create a unified 3D feature rich semantic map utilising MapAnything backbone and customised Mask2Former head. Above is an inference run on ScanNet++ dataset showcasing the model’s performance with respect to 3d reconstruction and panoptic segmentation from three different viewpoints of same scene.

multiple target views, and couples predictions through query propagation, ensuring that the same physical object receives consistent instance labels across different viewpoints.

We make the following contributions:

- 1) An integrated architecture combining 3D reconstruction features from the MapAnything backbone with trainable semantic features through a parallel DPT head design combined with a Mask2Former segmentation head.
- 2) A query propagation mechanism that leverages geometric correspondences to maintain instance consistency across multiple views.
- 3) A unified approach which does not require posed image inputs or costly test-time optimization strategies
- 4) Experimental validation on ScanNet++ dataset demonstrating that geometric features from reconstruction backbones can effectively guide panoptic segmentation when properly integrated.

II. RELATED WORK

A. Panoptic Scene Segmentation in 2D

Panoptic segmentation unifies instance and semantic segmentation by assigning every pixel both a class label and, where applicable, an instance identifier. The introduction of transformer-based architectures enabled more unified formulations: DETR[2] reformulated object detection as set prediction

using learnable queries, and Mask2Former [3] adapted this paradigm to panoptic segmentation through masked attention. In Mask2Former, a fixed set of object queries attends to multi-scale image features and predicts binary masks alongside class probabilities, with Hungarian matching used to assign queries to ground-truth instances during training. However, these methods operate on single 2D images and do not enforce consistency when the same scene is observed from multiple viewpoints.

B. Feed Forward Model for 3D Reconstruction

Classical approaches to reconstructing 3D scenes from images decompose the problem into sequential stages: feature detection and matching, camera pose estimation through bundle adjustment, and dense reconstruction via multi-view stereo. While effective, these modular pipelines accumulate errors across stages and require careful tuning for different scene types. Recent transformer-based methods have shown that direct regression of geometry from image pairs or sets can bypass these traditional stages. DUST3R[15] introduced dense point-map prediction from image pairs, learning to output per-pixel 3D coordinates directly. Extensions like MUST3R[1] and VGGT[14] scale this approach to arbitrary numbers of views by maintaining memory of previously seen frames through cross-attention mechanisms. MapAnything[5] and VGGT[14]

further demonstrate metric-accurate reconstruction without requiring posed inputs. These feed-forward models achieve strong geometric consistency by learning multi-view correspondence implicitly through large-scale pretraining, but output only geometric attributes without semantic or instance-level understanding.

C. Multi-View Panoptic Segmentation

Extending panoptic segmentation to 3D introduces challenges in representing instances across viewpoints and integrating semantic predictions with geometric reconstruction. A category of methods starts from multi-view RGB images and consolidates per-view 2D predictions into a unified 3D representation. These "lifting" approaches usually NeRF[10] or Gaussian Splatting[6] based typically run off-the-shelf 2D segmentation models independently on each frame, then fuse the results through volumetric rendering or explicit 3D scene representations. While lifting methods can aggregate inconsistent 2D predictions into cleaner 3D outputs, they share several limitations. First, they require known camera poses, either provided or estimated through separate structure-from-motion preprocessing. Second, they perform costly per-scene optimization to align 2D segmentations with 3D geometry, preventing real-time or feed-forward deployment.

A smaller body of work explores joint prediction of geometry and semantics within a single network. (i) PanSt3R[18] represents a step toward tighter integration, combining MUST3R's multi-view features with a dedicated Mask2Former decoder to predict panoptic labels in a single forward pass. By sharing learnable queries across all views and supervising them jointly, PanSt3R achieves multi-view consistent segmentation without test-time optimization. However, it processes a fixed set of keyframes (typically 50) and applies a separate quadratic optimization (QUBO) procedure to merge overlapping mask proposals into a final instance set. (ii) UNITE[8] adopts a different strategy, adding task-specific DPT heads for semantics, instances, and articulations on top of VGGT while keeping the backbone frozen. Each head outputs dense per-pixel features, and explicit multi-view consistency losses pull predictions for the same 3D point across different views toward agreement. This design avoids discrete mask merging but requires ground-truth depth and poses during training to establish cross-view correspondences.

In contrast, our approach builds on the observation that reconstruction backbones like MapAnything already learn rich multi-view features for geometric reasoning, and these features may transfer to semantic tasks if properly adapted. Rather than training separate models for geometry and segmentation, we introduce a trainable DPT adapter initialized from MapAnything's geometric weights to produce multi-scale features compatible with Mask2Former. By freezing the reconstruction backbone and training only the adapter and segmentation decoder, we aim to leverage pretrained geometric knowledge efficiently. For multi-view consistency, we design a query propagation mechanism that targets a query from a reference view to target views, encouraging the same object queries

to track instances across frames. This strategy integrates geometry and semantics within a single forward pass while avoiding both costly per scene optimization and separate post-processing steps.

III. METHOD

A. Overview

We propose an integrated architecture as shown in Figure 2 that augments a frozen 3D reconstruction backbone (MapAnything) with a trainable head (Mask2Former) for panoptic segmentation. Given a collection of N RGB images $\{I_1, I_2, \dots, I_N\}$ capturing a scene from different viewpoints, our system outputs per-view instance masks and class predictions that are consistent across overlapping frames. The pipeline consists of three main stages: (1) multi-view feature extraction through MapAnything's pretrained encoders, (2) transformation of geometric features into a multi-scale representation suitable for segmentation via a trainable DPT adapter, and (3) per-view panoptic prediction using a Mask2Former decoder with cross-view query supervision.

B. ScanNet++ Dataset Pre-Processing

a) *ScanNet++ structure.*: ScanNet++ is a large-scale indoor 3D dataset where each scene is a room or area captured with:

- **DSLR cameras:** High-resolution images at $\sim 6048 \times 4024$.
- **iPhone LiDAR:** Lower-resolution but with depth.
- **3D mesh reconstruction:** Via COLMAP/NerfStudio structure-from-motion.

Each scene contains:

- Undistorted DSLR images with known camera poses (camera-to-world 4×4 matrices).
- Camera intrinsics (focal length, principal point).
- A reconstructed 3D mesh with per-vertex semantic and instance labels.
- Split files defining train/val/test scenes.

b) *Panoptic annotations generation pipeline.*: Panoptic annotations are *not provided directly* by ScanNet++. They are generated through a two-stage pipeline:

1) Stage 1: 3D-to-2D Label Projection (`semantics_2d.py`)

ScanNet++ provides `semantics_2d.py` (an official toolkit script) that projects 3D mesh labels onto 2D images:

- Load the 3D mesh with per-vertex semantic class IDs and instance (object) IDs.
- For each DSLR image, render the mesh from that camera's viewpoint.
- Use z-buffering to determine which mesh face is visible at each pixel.
- Write per-pixel outputs as Instance ID tensor (PyTorch format) and Semantic class ID image (grayscale PNG).

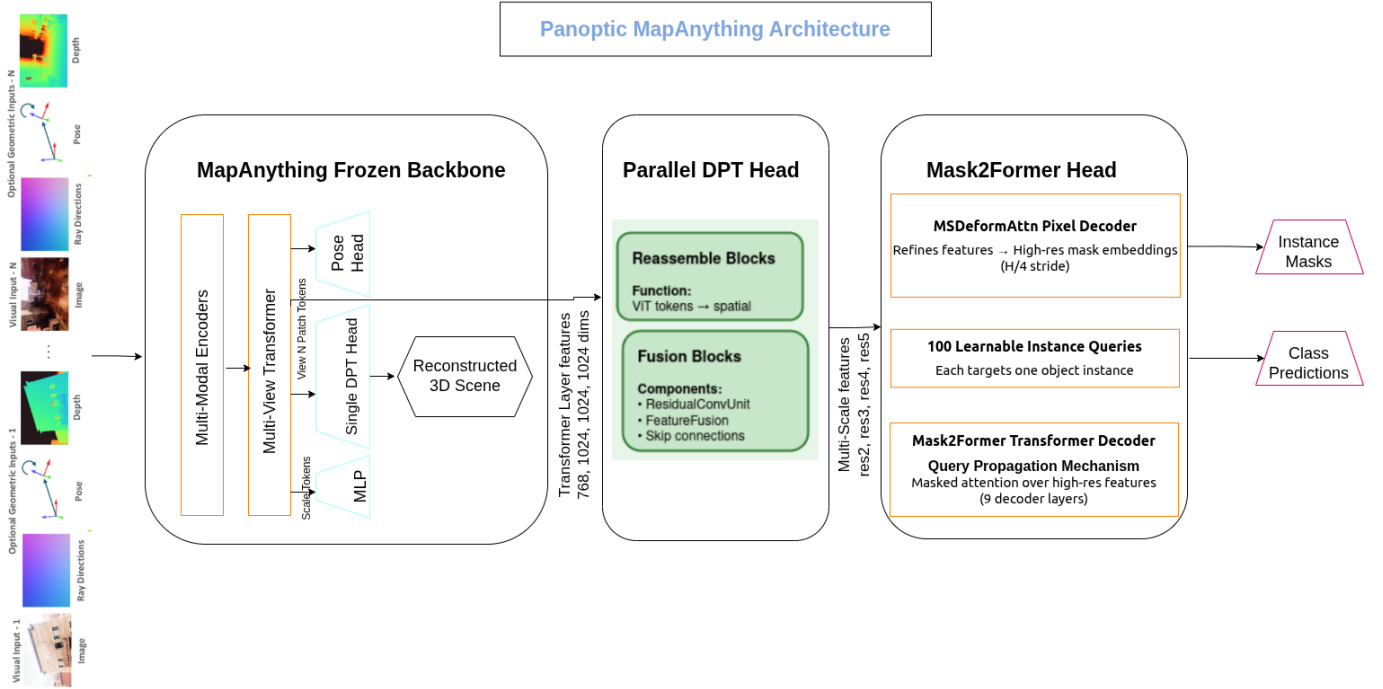


Fig. 2. **Architecture Overview.** Input - A set of RGB images. These are passed through the layers of the frozen MapAnything backbone which extracts the 3D features. We then pass the features from the multi-view transformer layer of MapAnything to our custom parallel DPT head which we train to get the multi-scale features which are then fed as input to the Mask2Former Panoptic head which predicts the masks and classes. The transformer decoder of our head also performs query propagation which aligns the queries for each object across the reference and target views.

2) Stage 2: Conversion to Detectron2 Format (convert_to_panoptic_format.py)

The custom conversion script transforms the raw per-pixel labels into the panoptic format expected by Detectron2/Mask2Former:

Panoptic ID encoding: The panoptic ID encoding is defined as $\text{panoptic_id} = \text{semantic_id} \times \text{LABEL_DIVISOR} + \text{instance_id}$, where $\text{LABEL_DIVISOR} = 10000$. (chosen to accommodate ScanNet++’s 2878 semantic classes with up to 9999 instances per class).

Create segments_info list (for each unique panoptic ID):

- `id`: The panoptic ID.
- `category_id`: The semantic class.
- `area`: Number of pixels.
- `isthing`: Whether the class is a “thing” (countable object) vs. “stuff” (amorphous region).
- `iscrowd`: 0 (no crowd annotations).

Panoptic Label Format:

- Save panoptic map as PNG (RGB encoding: $R + G \times 256 + B \times 65536$ for IDs > 65535 , or 16-bit grayscale for smaller IDs).
- Save `segments_info` as JSON.

C. Multi-view Mask2Former Decoder

a) Multi-View Dataset Mapper.: It is the critical component that transforms raw dataset records into training-ready multi-view batches. For each training sample,

- 1) **Scene Loading:** Load camera data (NerfStudio transforms or COLMAP fallback) with caching for efficiency.
- 2) **View Selection:** Select N diverse views using maximum overlap sampling.
- 3) **Consistent Resizing:** Sample a single target size for all views (from $[640, \dots, 800]$) to ensure identical dimensions.
- 4) **Image Processing:** Read RGB image, apply resize transform, pad to common dimensions (value 128), and convert to CHW tensor.
- 5) **Panoptic Annotation Loading:** Read panoptic PNG, decode RGB to panoptic ID via `rgb2id()`, pad with `ignore_label` (255), and load annotations.

D. Multi-View Feature Extraction

We use MapAnything as our geometric foundation, keeping all of its weights frozen throughout training. Each input image I_i is independently processed by a pretrained DINOv2 ViT-L encoder, producing 768-dimensional semantic features at a spatial resolution of $H/14 \times W/14$ where H and W denote image height and width. These per-view features are then passed to MapAnything’s Multi-View Info-Sharing Trans-

former, which exchanges information across frames through cross-view attention. The transformer outputs refined 1024-dimensional features for each view that encode both local appearance and global geometric context derived from frame-to-frame correspondences.

By freezing the backbone, we preserve MapAnything’s learned ability to reason about multi-view geometry while avoiding the computational cost and potential instability of fine-tuning the entire reconstruction network. This design choice is motivated by the hypothesis that geometric features which must already capture spatial relationships and cross-view correspondences to succeed at depth prediction provide a suitable foundation for semantic understanding.

E. Panoptic DPT Adapter

The 1024-dimensional features output by MapAnything’s transformer are structured as patch tokens rather than the multi-scale feature pyramid expected by Mask2Former’s pixel decoder. To bridge this representation gap, we introduce a trainable Panoptic DPT head that transforms the single-resolution tokens into four feature maps at different spatial scales (res2, res3, res4, res5 with strides 4, 8, 16, and 32 respectively).

The DPT adapter as shown in Figure 2 consists of four sequential stages. First, **reassemble blocks** rearrange the flattened ViT tokens back into spatial grids, preparing them for convolutional operations. Second, **projection layers** adjust the channel dimensions to match the target output sizes (96, 192, 384, and 768 channels for res2 through res5). Third, **resize layers** use bilinear upsampling or downsampling to produce feature maps at the four required spatial resolutions. Fourth, **fusion blocks** apply RefinNet-style bottom-up fusion with residual connections and skip connections to gradually combine information across scales. Finally, parallel 1×1 convolutions at each resolution produce the final multi-scale output.

Crucially, we initialize the DPT adapter using weights from MapAnything’s geometric DPT head, which was originally trained to regress depth maps. This initialization provides the adapter with a strong prior for spatial reasoning and multi-scale feature fusion. During training, the adapter learns to specialize these geometric features for panoptic segmentation while the frozen backbone continues to provide consistent multi-view representations.

The multi-scale features from the DPT adapter are passed to a Mask2Former architecture consisting of a pixel decoder and a transformer decoder.

a) Pixel Decoder.: A six-layer MSDeformAttn (multi-scale deformable attention) module refines the four input feature levels (res2–res5) to produce high-resolution mask embeddings at stride 4, with 256 output channels. This component upsamples and fuses information across scales to construct detailed spatial features suitable for segmenting fine object boundaries.

b) Transformer Decoder.: We initialize 100 learnable object queries, each representing a potential instance in the

scene. These queries are processed through nine transformer decoder layers, where each layer performs two operations: (1) masked cross-attention between queries and the pixel decoder’s mask embeddings, allowing each query to attend to relevant spatial regions, and (2) self-attention among queries to enforce distinctness between different instance proposals.

After refinement, each query produces two outputs. A **mask prediction** is obtained by computing the dot product between the query’s mask embedding and the high-resolution features, followed by a sigmoid activation to produce a binary mask. A **class prediction** is generated by passing the query’s class embedding through a linear classifier with a final softmax over all the panoptic classes.

c) Query Propagation Transform Decoder.: Query propagation as shown in Figure 3 is the central mechanism for achieving cross-view consistent panoptic segmentation. Without it, each view would independently produce 100 queries with no correspondence. We modified the transform decoder for target views in a way where it accepts reference view’s refined query embeddings (Q_{ref}) instead of learnable queries. Query #K in the reference view automatically becomes Query #K in all target views, the decoder then refines these queries using the target view’s features. Therefore, if the decoder learns to segment correctly, query #K will predict the same semantic entity across views. This preserves query identity across views: query #5 predicting “chair” in the reference view will also predict “chair” in the target views thereby enforcing structural consistency between all the views of the same scene.

d) Modified Hungarian Matcher.: The Hungarian matcher solves a bipartite matching problem: assign each of the 100 predicted queries to the ground truth segments in a way that minimizes total cost. The cost is a weighted sum of: $cost = \lambda_{cls} \cdot C_{class} + \lambda_{mask} \cdot C_{mask} + \lambda_{dice} \cdot C_{dice}$. Where:

- C_{class} : Classification cost (negative log probability of the correct class)
- C_{mask} : Binary cross-entropy between predicted and GT masks (sampled at sparse points)
- C_{dice} : Dice coefficient cost for mask overlap quality

We modified the implementation to avoid NaN issues while computing losses by clamping `pred_logits` to $[-50, 50]$. Additionally, `pred_masks` were soft-clamped using tanh compression beyond ± 10 .

e) Multi-view Loss.: Computes the final training losses after Hungarian matching.

After the matching step assigns queries to ground truth (GT) segments, `SetCriterion` computes three specific losses:

- **Cross-Entropy Loss (`loss_ce`)**: Classification loss over all queries (matched + unmatched). Unmatched queries are trained to predict the “no object” class ($\emptyset$), weighted by `no_object_weight` (0.1) to avoid overwhelming the total loss.
- **Binary Mask Loss (`loss_mask`)**: Per-pixel sigmoid binary cross-entropy (BCE) between predicted mask logits and GT binary masks. This is computed at *sampled points* (12,544 points by default) for memory efficiency, using

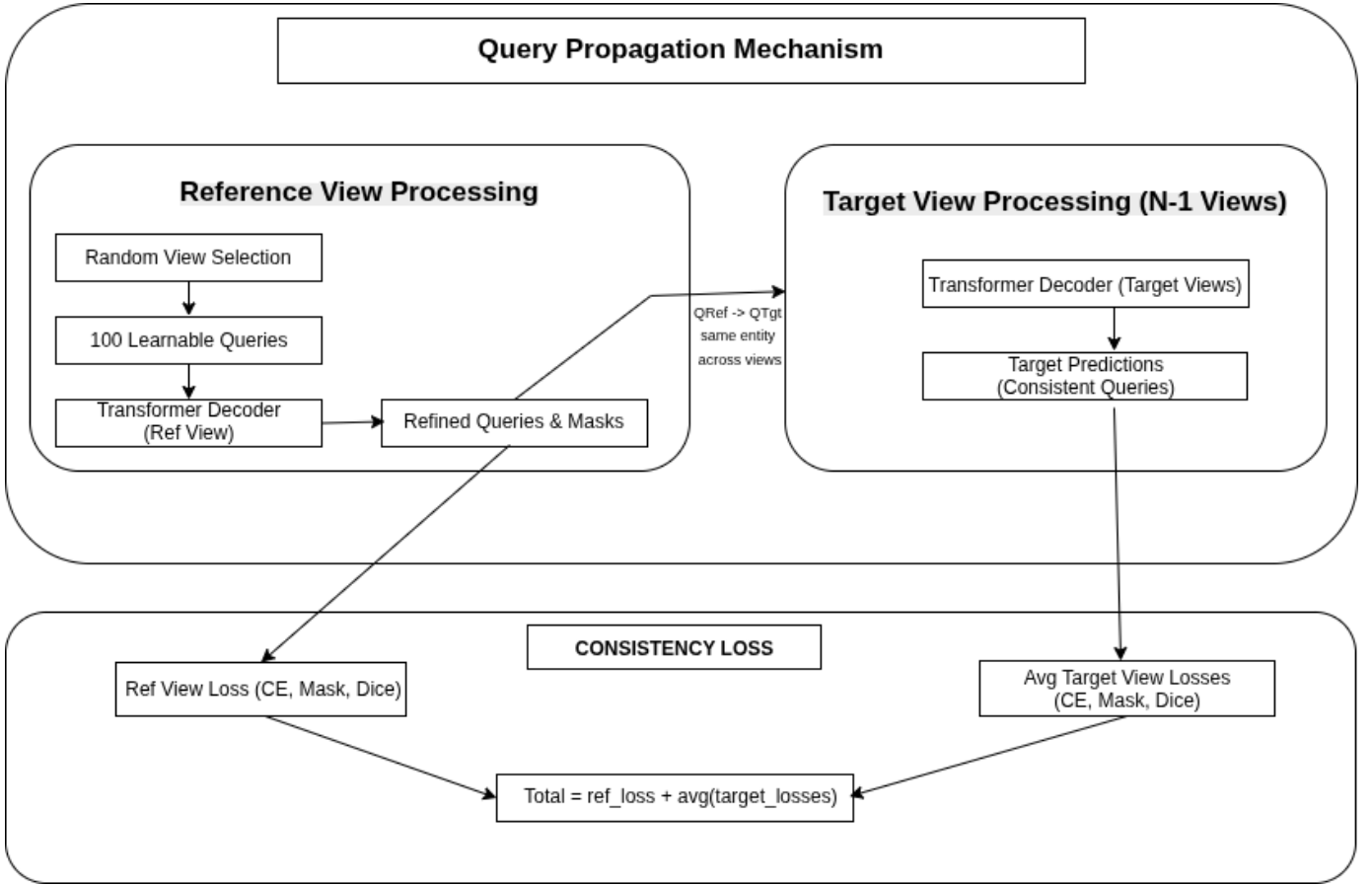


Fig. 3. **Query Propagation in detail.** Aligns the queries for each object across the reference and target views using the reference+target view losses.

oversampling (3 $\times$) with importance sampling to focus on uncertain regions.

- **Dice Loss (loss_dice):** Measures overlap quality between the predicted mask P and GT mask G . The loss is defined as:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2|P \cap G|}{|P| + |G|} \quad (1)$$

Dice loss is critical for balanced training because it is scale-invariant (unlike BCE, it is not dominated by the large number of background pixels).

Deep supervision. Losses are computed at *all 9 intermediate decoder layers* (plus the final layer), rather than just the last layer. This provides a stronger gradient signal to early layers and prevents vanishing gradients in the deep transformer architecture.

Following loss weights are configured CE = 2.0, Mask = 5.0, Dice = 5.0. This is inline with Mask2Former Panoptic Segmentation training guidelines[3].

The losses are constructed for both reference and target views. The total multi-view consistency loss is a combination of these components, where target view losses utilize propagated queries (enabled after a warmup phase).

The total loss is a weighted combination of reference and target losses:

$$\mathcal{L}_{\text{total}} = w_{\text{ref}} \cdot \mathcal{L}_{\text{ref}} + w_{\text{target}} \cdot \frac{1}{N-1} \sum_{v \neq \text{ref}} \mathcal{L}_v + \mathcal{L}_{\text{logit_reg}} \quad (2)$$

Where:

- $w_{\text{ref}} = 1.0$: Reference view at full weight.
- $w_{\text{target}} = 0.67/N$: Target views averaged then scaled.

Target losses are first averaged across target views to prevent gradient explosion from multi-view accumulation, which are then multiplied by w_{target} . Detached copies of target losses are stored as `loss_ce_target`, `loss_mask_target`, and `loss_dice_target` for monitoring (no gradient flow through these). Both reference and averaged target losses are summed into the same loss keys (`loss_ce`, `loss_mask`, `loss_dice`), so a single backward pass flows gradients through all views.

Mask Logit Regularization An additional regularization loss penalizes extreme mask logit magnitudes to prevent divergence, acts as a "soft spring" pulling logits toward a safe range.

$$\mathcal{L}_{\text{logit_reg}} = \lambda_{\text{reg}} \cdot \text{mean}(\max(0, |\text{logit}| - \tau)^2) \quad (3)$$

Where:

- $\tau = 10.0$: Threshold (no penalty within $[-10, 10]$).
- $\lambda_{\text{reg}} = 0.01$: Small weight (just prevents runaway).

IV. EXPERIMENTS

A. Dataset Preparation and Class selection

Instead of using the full 856 scenes from ScanNet++, we selected a smaller subset consisting of the first 144 scenes for training and 48 scenes for validation, maintaining a 3:1 ratio. Preprocessing the dataset required significant time, specifically rasterizing 3D mesh annotations to generate 2D panoptic labels for each scene. Additionally, for the warped attention mechanism, we had to pre-render depth information using the ScanNet++ toolkit. Consequently, we decided to proceed with this smaller subset to first construct a stable training pipeline, with the intention of scaling the dataset later if time permitted.

While ScanNet++ defines 2,878 semantic classes, our training subset of 144 scenes contained only 254 distinct classes. Using a 2,879-way softmax where over 90% of classes have zero training examples would render the classification cost matrix meaningless, diluting the Cross-Entropy (CE) loss gradient by approximately $11\times$ across unseen classes. Therefore, we configured the number of training classes to 254 based on an analysis of the panoptic annotations in our chosen subset. This resulted in a smaller `class_embed` tensor of size $[256, 255]$ instead of $[256, 2879]$, thereby saving parameters and memory. Following this change, we observed a significant improvement in detecting “thing” classes and a reduction in false positives for object detection.

B. Farthest Point Sampling and Maximum Overlap Aware View Selection

We experimented with two approaches for reference and target view selection: Farthest Point Sampling and Maximum Overlap-Aware View Selection. With the former approach, we selected views that had minimal overlap between scenes. This resulted in query propagation often failing, leading to the misclassification of objects in the target views, which in turn affected the RQ (Recognition Quality) between views of the same scene. To mitigate this issue, we later switched to the Maximum Overlap-Aware algorithm, which selects views with high visual overlap (approximately 50–70%) based on a configured camera distance range and the likelihood of shared regions between views based on viewing direction similarity. With this new approach, the query propagation mechanism improved comparatively, and the misclassification rate was significantly reduced.

C. With Warped Attention Mask

We initially experimented with using a warped attention mask alongside query propagation in the transformer decoder for the target view. For this mechanism to function, pre-rendered depth information is required to warp reference masks to the target view’s image plane, serving as a geometrical prior. However, the results were not convincing; the masks were improperly aligned, containing numerous fragmented

patches, and the predictions were frequently misclassified compared to the Ground Truth (GT).

The concept behind this warped attention mask mechanism is to act as a “spatial bridge,” guiding a query on where to look within the target view’s image plane. Without a warped mask acting as a prior, a propagated query from the reference view would attempt to attend to features at its original (reference) coordinates. In the target view, these coordinates might now correspond to empty space or a different object. We also implemented alpha blending, mixing geometric warped masks from the reference view with self-predicted masks, decaying linearly from $\alpha = 1.0$ (layer 0) to $\alpha = 0.0$ (layer 9) to guide the mask embeddings of the target views based on these warped features. Despite this, the inference results were unsatisfactory, indicating that a localized patch-based approach to finding corresponding queries from the reference view often fails when the views represent significantly different viewpoints of the same scene.

D. Without Warped Attention Mask

We disabled the warped attention mask mechanism and trained our model exclusively with Query Propagation between the reference and target views. The multi-view transformer in the MapAnything backbone already shares information across all N views simultaneously using alternating attention. Consequently, the features are already spatially aware of object locations within the scene, irrespective of viewpoint differences. We decided to rely on this implicit feature alignment to guide the identification of propagated queries in the target views. As a result, the transformer decoder now attends to the entire global context of the image features to locate the object represented by the query. The results from table I shows a significant improvement in object identification compared to the warped attention mask approach during our initial training of the model for 22,500 iterations on a single A100 GPU.

E. Varying the loss weights and hyperparameters

We experimented with adjusting the weights for the following losses: Cross-Entropy (CE), Mask, and Dice. Initially, we set all these values to 5.0, matching the configuration used for single-view training on the COCO dataset. However, this setup proved ineffective for our ScanNet dataset; the mask loss collapsed, causing the model to predict nothing as the mask logits remained negative. Consequently, we adjusted the loss weights to CE = 2.0, Mask = 5.0, and Dice = 5.0, in line with the Mask2Former panoptic segmentation training guidelines. This configuration resulted in much better loss convergence compared to other combinations. We achieved a minimum loss of 90 after over 30,000 iterations when training on a single GPU.

Additionally, we modified our `IMS_PER_BATCH` setting to correspond to the number of scenes rather than the number of images. We configured this to 4, representing 4 scenes, where each scene consists of 3 views (one reference view and two target views). This adjustment resulted in significantly more stable gradients during single-GPU training. We clipped

the gradients to a maximum norm of 5.0. In our experiments, applying a more restrictive clipping value of 1.0 caused the loss to oscillate, indicating that the model was failing to learn.

F. Transfer Learning

We utilized transfer learning from a baseline model trained on the COCO dataset, which performed single-view panoptic segmentation using a MapAnything backbone. The real challenge was that training from scratch (without transfer learning) with 254 classes proved extremely difficult. The mask logits stagnated at -12 (sigmoid ≈ 0.000006), indicating the model predicted “no mask” everywhere and could not escape this local minimum. Consequently, we initialized our multi-view panoptic segmentation model with the weights from the baseline model. This prevented the weights from collapsing due to exploding gradients at the start of training, as they were no longer randomly initialized. When transfer learning was enabled, the MapAnything backbone loaded its pre-trained weights (from its own checkpoint), while the Panoptic DPT, pixel decoder, and Mask2Former transformer decoder were initialized with the baseline model weights. The only exception was the `class_embed` head, which required different dimensions (ScanNet++ 254 classes compared to the 133 classes in COCO). With these components pre-trained, the mask logits started near 0 rather than as random noise, allowing the model to immediately begin learning multi-view consistency instead of basic segmentation from scratch.

G. Per-Component Learning Rates

We implemented differential learning rates for the trainable components, specifically the Panoptic DPT Adapter Head, pixel decoder, and Mask2Former decoder with query propagation. The learning rates were assigned as follows: DPT ($1e-5$), `embed_heads` ($5e-5$, with a higher weight decay of 0.05), `semseg` ($5e-5$), and `query_propagation` ($0.5e-5$). The DPT head was initialized from a pre-trained geometric DPT and required gentle fine-tuning. The embedding heads (`mask_embed`, `class_embed`) were the most prone to NaN errors and benefited from higher regularization. Query propagation, being a new component, tended to diverge easily. Consequently, defining parameter groups with distinct learning rates prevented the DPT overfitting, embedding head divergence, and query propagation instability that occurred with a uniform learning rate.

H. Multi-Gpu Training

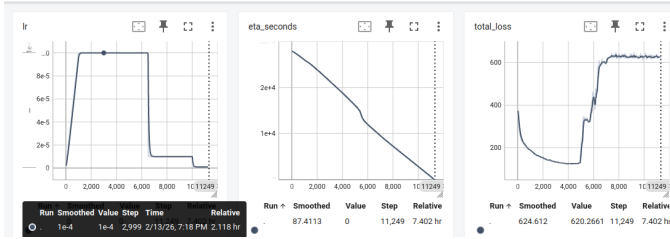


Fig. 4. Multi-gpu training shows an abnormal upward loss curve trend

We attempted to train our model using a multi-GPUs setup consisting of 4 NVIDIA A100 and 8 NVIDIA A40 GPUs; however, securing these resources on the Marvin cluster was often challenging due to limited availability. During these sessions, we observed Distributed Data Parallel (DDP) synchronization issues that arose when one of the GPU’s computed a NaN loss. To mitigate this, we implemented a strategy to detect invalid batches and skip the backpropagation step for them. Following this modification, training proceeded stably for approximately 10,000 iterations, reaching a minimum loss of 124. However, beyond this point, the loss began to diverge and followed an upward trend, as shown in Figure 4. This instability appears specific to the multi-GPU regime, warranting further investigation to identify the root cause.

V. RESULTS AND EVALUATION

A. Training Setup

We present our results based on single-GPU training using an NVIDIA A100, which ran for approximately 30,000 iterations with the loss converging around 90 as shown in Figure 5. For this experiment, we set `IMS_PER_BATCH` to correspond to the number of scenes rather than the number of images. We configured this to 4, representing 4 scenes, where each scene consists of 3 views (one reference view and two target views). Additionally, we adjusted the loss weights to Cross-Entropy (CE) = 2.0, Mask = 5.0, and Dice = 5.0, in line with the Mask2Former panoptic segmentation training guidelines. These adjustments resulted in significantly more stable gradients and successful loss convergence during single-GPU training.

We performed inference on a single scene from the validation set and conducted a full evaluation on all 48 scenes within the validation set. Due to time constraints, we were unable to evaluate the model on the ScanNet++ test scenes. Preprocessing the test set to retrieve 2D panoptic Ground Truth (GT) labels from the 3D mesh annotations requires approximately 8 hours. Consequently, we restricted our evaluation of the trained multi-view panoptic segmentation model to the validation set.

We used the CSV logging for the loss metrics and tensorboard for monitoring those metrics as the training progressed. Following memory optimisations were performed: reduced the image resolution $384\text{-}480 \times 640$ (from DSLR 4000×6000) and AMP (fp16) enabled for trainable components.

B. Inference Setup

We loaded N views per scene, employing random selection for the reference view and overlap-aware selection for the target views. These N views were processed through the MapAnything backbone, where view 0 generated learnable queries. These queries were then propagated to the target views ($1 \dots N-1$) to establish consistent query representations across different viewpoints of the same scene. Subsequently, we post-processed each view’s prediction using overlap merging and confidence thresholding to obtain per-view predictions: semantic classes (Q queries), instance masks ($[Q, H, W]$), and

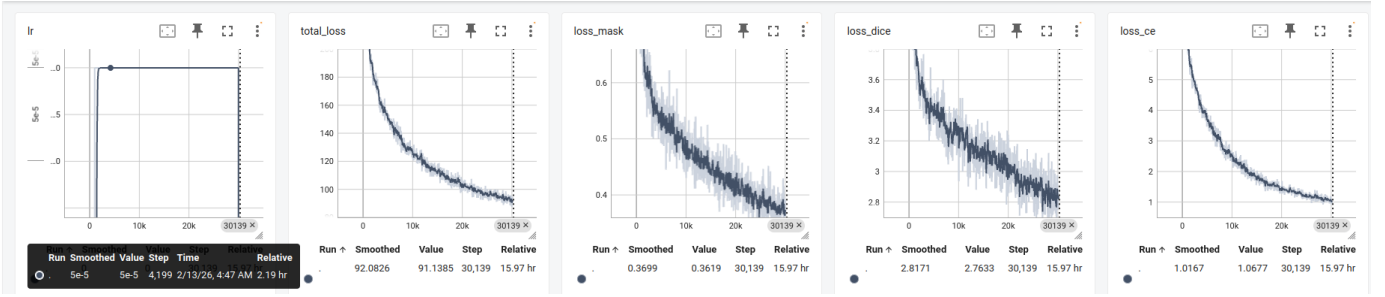


Fig. 5. Single-gpu training shows an downward loss curve trend

confidence scores ($[Q]$). We compare these 2D predictions against the 2D instance and semantic Ground Truth (GT) annotations, as illustrated in Figure 6.

Building on the approach from our midterm presentation with the single-view baseline, we projected the multi-view model outputs into 3D space. We utilized MapAnything’s 3D point generation(pts3d) directly to reconstruct the scene from the input images, as shown in Figure 1. We then segmented this 3D point cloud based on our 2D predictions, assigning a higher weight ($1.5\times$) to predictions from the reference view compared to the target views. To fuse points from multiple views, we employed a voxel voting mechanism. The 3D space was discretized into voxels of size 0.02 m (configurable), and points from all views were aggregated for each voxel. Points originating from the reference view were weighted by a `ref_view_weight` factor of 1.5. Points falling below a combined confidence threshold of 0.5 were discarded; this

score was derived from the product of the MapAnything confidence and the Mask2Former confidence score. Instance ID and class for each voxel are decided by summing the vote weights per unique label, the label with the highest total weight wins. Consistent RGB colors from the 2D panoptic predictions were applied to the classes in the 3D segmented point cloud, as depicted in Figure 7.

C. Evaluation Metrics

The evaluation pipeline mirrors the training forward pass, utilizing the same multi-view backbone with cross-view attention and query propagation across reference and target views. Initially, we process all 48 scenes in the validation set, performing preprocessing steps such as image resizing, padding, and loading the corresponding panoptic Ground Truth (GT) labels.

For each scene, we select N views using overlap-aware sam-

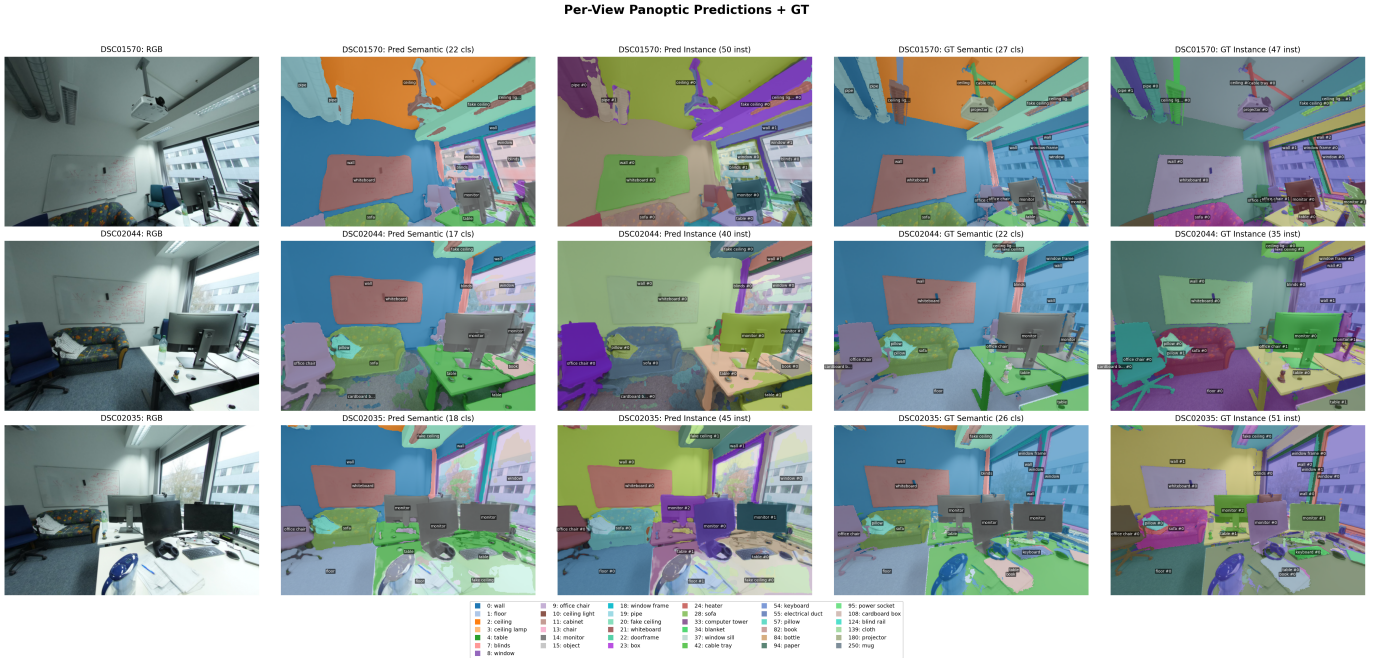


Fig. 6. Comparison of predicted results for each input views vs the Ground truth annotations obtained as part of inference.

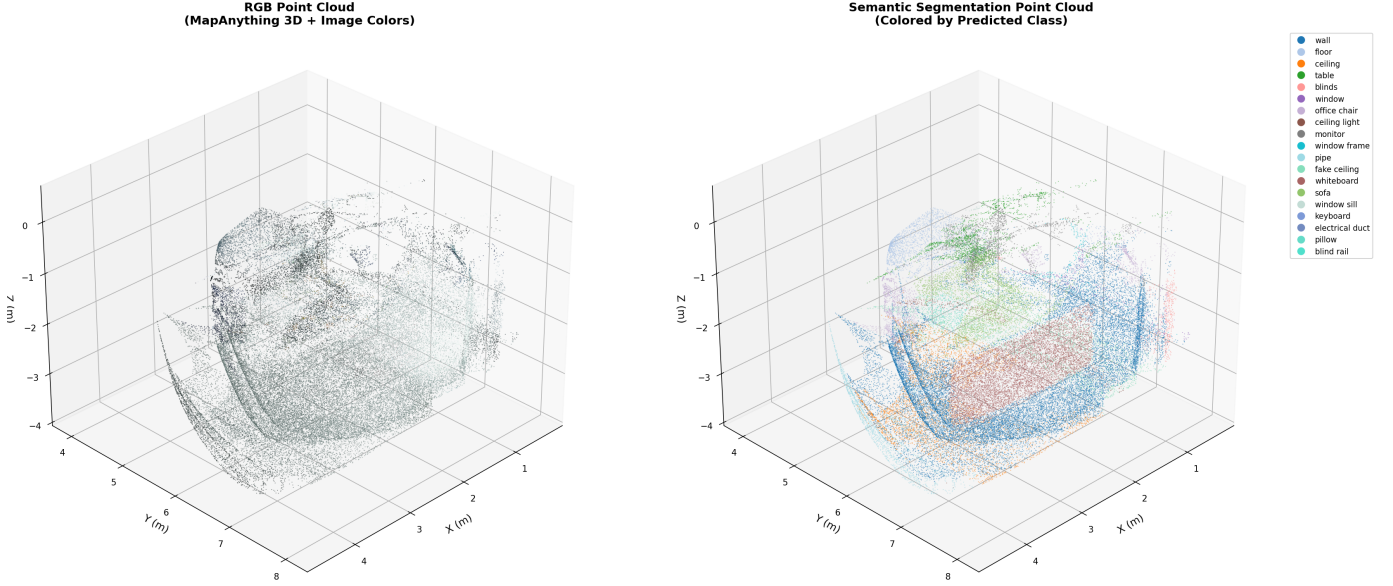


Fig. 7. Segmentation performed by our model on top of MapAnything’s 3D reconstructed point cloud

pling, ensuring that views have camera poses within a distance of 0.3–2.0 m to facilitate meaningful multi-view reasoning, and are paired with valid GT annotations. We then execute full multi-view inference for each scene. Consistent with our training strategy, we utilize 3 views (one reference and two target views) for evaluation. Once predictions are generated, we compare each view’s output against its respective GT. We compute per-view metrics including Panoptic Quality (PQ), Recognition Quality (RQ), and Segmentation Quality (SQ) using the official `panopticapi.evaluation.pq_compute()` function for each view independently. Subsequently, we calculate the average PQ, PQ variance, and PQ standard deviation within the scene.

We also compute the Global PQ, which is a macro-average across all 48 scenes; this provides insight into the backbone quality (cross-view features) as distinct from the decoder quality (per-view predictions). The per-view PQ variance reveals whether the model performs consistently across different viewpoints or relies heavily on the quality of the reference view. Additionally, we compute cross-view metrics to directly measure whether query propagation successfully maintains entity identity across viewpoints. For each scene in the validation set, we evaluate: (1) **Class Consistency**, which measures if Query k predicts the same semantic class across all views; (2) **Query Similarity**, which assesses the cosine similarity of query embeddings across views to validate feature alignment; and (3) **Mask IoU**, which quantifies the spatial overlap of predictions (a metric primarily useful when warped attention is enabled). By evaluating on 3 views per scene rather than all available views, we balance computational cost with statistical significance.

D. Results from our training

As previously described, we conducted an evaluation on the 48 scenes comprising the validation set for our multi-view panoptic segmentation model. This model was trained on a subset of 144 scenes over 30,000 iterations, with the training loss converging to approximately 90. For the evaluation, we utilized a 3-view configuration (one reference and two target views), mirroring the setup used during training.

To generate the final panoptic predictions, we applied specific post-processing thresholds: an Intersection over Union (IoU) overlap threshold of 0.6 and a mask threshold of 0.6. These parameters control the greedy overlap merging process, where predictions are sorted by confidence, and pixels are assigned to the highest-confidence non-overlapping segment. The chosen IoU threshold of 60% is stricter than the standard COCO threshold of 0.5; this increased stringency is intentionally selected to produce cleaner segmentations suitable for indoor scenes, where object boundaries are often more defined. While the overlap threshold governs instance merging, the mask threshold controls binary mask generation, ensuring that only high-confidence predictions contribute to the final segmentation map.

Listing 1. Evaluation Summary Metrics

```
{
  "PQ": 5.957468242741876,
  "SQ": 11.344218016411766,
  "RQ": 8.34326670551303,
  "PQ_th": 5.914064341563311,
  "SQ_th": 11.422901486117809,
  "RQ_th": 8.277817982465045,
  "PQ_st": 6.227423867736538,
  "SQ_st": 11.553993336530587,
  "RQ_st": 8.779810027436957,
  "avg_per_view_pq": 7.9330615217531575,
```



```

"pq_variance": 15.609642710879436,
"pq_std": 3.036067423572238,
"PQ_view_0": 8.077632728198061,
"PQ_view_1": 7.5457719271746315,
"PQ_view_2": 8.175779909886783,
"class_consistency": 0.2206250000745058,
"mask_iou": 0.048193883736126326,
"query_cosine_similarity": 0.6075105068998204,
"num_views_per_scene": 3,
"num_scenes_evaluated": 48,
"num_scenes_skipped": 0,
"num_scenes_total": 48
}

```

The model achieved a Panoptic Quality (PQ) of 5.96, with Segmentation Quality (SQ) of 11.34 and Recognition Quality (RQ) of 8.34 across 254 fine-grained semantic categories. The low RQ relative to SQ indicates that the primary bottleneck is segment detection rather than mask precision, i.e. when segments are matched, the IoU quality is reasonable, but the model fails to detect the majority of ground-truth segments.

Per-view analysis demonstrates consistent performance across the reference view (PQ = 8.08) and target views (PQ = 7.55, 8.18), confirming that the query propagation mechanism does not degrade prediction quality in the target views. However, cross-view consistency metrics indicate that the query propagation has not yet fully converged. Specifically, class consistency reaches only 22.1%, and query cosine similarity is limited to 0.608.

Scene-level analysis reveals high variance ($\sigma = 3.04$), with top-performing scenes achieving a per-view PQ of up to 21.8, while several others score near zero. The query embeddings exhibit moderate similarity across views (0.61) compared to the target threshold of 0.85. This suggests that while the transformer decoder partially preserves propagated query features, it significantly modifies them within each view. The 9-layer decoder, which utilizes cross-attention over different pixel features in each view, transforms the queries substantially. At 30,000 iterations (with 25,000 involving propagation), the decoder has not yet learned to robustly maintain semantic identity via the propagated query embeddings. Consequently, while the queries are utilized, each view’s cross-attention mechanism heavily re-interprets them based on local pixel features.

Per-scene analysis highlights significant disparities: scene `bde1e479ad` achieves a class consistency of 61% (best), whereas scene `45b0dac5e3` reaches only 11% (worst). The results from the best-performing scene suggest that the mechanism is viable but has not yet generalized across the dataset. The detailed evaluation metrics are shown in the Listing 1.

These results represent an early-stage checkpoint of a novel architecture combining a frozen MapAnything backbone with Mask2former decoder utilising query propagation on 254 classes. Although these metrics do not yet surpass our midterm baseline established with the single-view model on COCO dataset (133 classes), they represent a significant improvement over the results that were obtained using the warped attention mechanism as shown in table I. This indicates that leveraging global context is far more effective than a patch-level search

when identifying the propagated queries in the target frame.

TABLE I
COMPARISON OF PANOPTIC SEGMENTATION PERFORMANCE:
SINGLE-VIEW BASELINE (COCO) VS. MULTI-VIEW MODEL
(SCANNet++)

Metric Category	PQ	SQ	RQ
Single-View Baseline (COCO Val2017)			
Overall	40.0	78.7	48.4
Things (80 classes)	42.2	78.8	50.8
Stuff (53 classes)	36.8	78.6	44.8
Multi-View Model (ScanNet++ Val) – No Warped Attention			
Overall	5.96	11.34	8.34
Things (PQ _{th} , SQ _{th} , RQ _{th})	5.91	11.42	8.28
Stuff (PQ _{st} , SQ _{st} , RQ _{st})	6.23	11.55	8.78
Multi-View Model (ScanNet++ Val) – With Warped Attention			
Overall	2.27	6.53	3.40
Things (PQ _{th} , SQ _{th} , RQ _{th})	0.69	2.53	1.03
Stuff (PQ _{st} , SQ _{st} , RQ _{st})	5.17	13.15	7.70

VI. CONCLUSION AND FUTURE WORK

In this project, we demonstrated that a 2D panoptic segmentation head, such as Mask2Former, can be effectively retrofitted onto a frozen 3D reconstruction backbone like MapAnything. This integration paves the way for a low-overhead, single-forward-pass 3D panoptic segmentation model capable of constructing metric-level accurate semantic maps for mobile robotics navigation.

Although our proof-of-concept model did not achieve state-of-the-art performance during initial experimentation, we strongly believe that our approach has the potential to scale effectively and yield superior results given a multi-GPU training setup and a longer iteration schedule. This can be attributed to the improved class consistency and higher Panoptic Quality (PQ) metrics observed in specific scenes such as `bde1e479ad`, even with the limited checkpoint model used in our experiments. However, due to time constraints, we were unable to conduct further experiments to fully realize this performance potential.

For future work, we believe there is a lot of scope to improve this method further by implementing the following upgrades.

- 1) Fix the DDP synchronisation issues with multi-gpu setup and train for more iterations for better loss convergence.
- 2) Train on the full ScanNet++ dataset on a multi-node GPU setup for more iterations.
- 3) Try unfreezing certain layers of the mapanything backbone for training.
- 4) Experimenting with Open Vocabulary approaches like SigLip.

ACKNOWLEDGMENTS

We would like to thank our supervisors, Prof. Dr. Hermann Blum and Sami Azirar, MSc for providing valuable guidance and support during this project.

REFERENCES

- [1] Yohann Cabon, Lucas Stoffl, Leonid Antsfeld, Gabriela Csurka, Boris Chidlovskii, Jerome Revaud, and Vincent Leroy. Must3r: Multi-view network for stereo 3d reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1050–1060, June 2025.
- [2] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *Computer Vision – ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part I*, page 213–229, Berlin, Heidelberg, 2020. Springer-Verlag. ISBN 978-3-030-58451-1. doi: 10.1007/978-3-030-58452-8_13. URL https://doi.org/10.1007/978-3-030-58452-8_13.
- [3] Bowen Cheng, Ishan Misra, Alexander G. Schwing, Alexander Kirillov, and Rohit Girdhar. Masked-attention mask transformer for universal image segmentation. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1280–1289, 2022. doi: 10.1109/CVPR52688.2022.00135.
- [4] Fangzhou Hong, Hui Zhou, Xinge Zhu, Hongsheng Li, and Ziwei Liu. Lidar-based panoptic segmentation via dynamic shifting network. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13090–13099, June 2021.
- [5] Naveen Keetha, Norman Müller, Johannes Schönberger, Lorenzo Porzi, Yinda Zhang, Tobias Fischer, Arno Knapitsch, Daniel Zauss, Ethan Weber, Nuno Antunes, Jonathon Luiten, Marc Lopez-Antequera, Samuel Rota Bulò, Christian Richardt, Deva Ramanan, Sebastian Scherer, and Peter Kotschieder. Mapanything: Universal feed-forward metric 3d reconstruction. *arXiv preprint arXiv:2509.13414*, 2025.
- [6] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4), July 2023. ISSN 0730-0301. doi: 10.1145/3592433. URL <https://doi.org/10.1145/3592433>.
- [7] Alexander Kirillov, Kaiming He, Ross Girshick, Carsten Rother, and Piotr Dollar. Panoptic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.
- [8] Sebastian Koch, Johanna Wald, Hidenobu Matsuki, Pedro Hermosilla, Timo Ropinski, and Federico Tombari. Unified semantic transformer for 3d scene understanding. *arXiv preprint arXiv:2512.14364*, 2025.
- [9] Jinke Li, Xiao He, Yang Wen, Yuan Gao, Xiaoqiang Cheng, and Dan Zhang. Panoptic-phnet: Towards real-time and high-precision lidar panoptic segmentation via clustering pseudo heatmap. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11809–11818, June 2022.
- [10] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: representing scenes as neural radiance fields for view synthesis. *Commun. ACM*, 65(1):99–106, December 2021. ISSN 0001-0782. doi: 10.1145/3503250. URL <https://doi.org/10.1145/3503250>.
- [11] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 12179–12188, October 2021.
- [12] Ryan Razani, Ran Cheng, Enxu Li, Ehsan Taghavi, Yuan Ren, and Liu Bingbing. Gp-s3net: Graph-based panoptic sparse semantic segmentation network. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 16076–16085, October 2021.
- [13] Kshitij Sirohi, Rohit Mohan, Daniel Büscher, Wolfram Burgard, and Abhinav Valada. Efficientlps: Efficient lidar panoptic segmentation. *IEEE Transactions on Robotics*, 38(3):1894–1914, 2022. doi: 10.1109/TRO.2021.3122069.
- [14] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5294–5306, June 2025.
- [15] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. Dust3r: Geometric 3d vision made easy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20697–20709, June 2024.
- [16] Shuangjie Xu, Rui Wan, Maosheng Ye, Xiaoyi Zou, and Tongyi Cao. Sparse cross-scale attention network for efficient lidar panoptic segmentation. In *AAAI-22 Technical Tracks 3*, Proceedings of the 36th AAAI Conference on Artificial Intelligence, AAAI 2022, pages 2920–2928. Association for the Advancement of Artificial Intelligence, June 2022. doi: 10.1609/aaai.v36i3.20197. Publisher Copyright: Copyright © 2022, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.; 36th AAAI Conference on Artificial Intelligence, AAAI 2022 ; Conference date: 22-02-2022 Through 01-03-2022.
- [17] Chandan Yeshwanth, Yu-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *Proceedings of the International Conference on Computer Vision (ICCV)*, 2023.
- [18] Lojze Zust, Yohann Cabon, Juliette Marrie, Leonid Antsfeld, Boris Chidlovskii, Jerome Revaud, and Gabriela Csurka. Panst3r: Multi-view consistent panoptic segmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5856–5866. Naver Labs Europe, 2025.